

รายงานผลการทดสอบ PoC: การป้องกันพารามิเตอร์ ArduPlane

สรุปสั้น

ผลการทดสอบ PoC นี้คือ ผ่าน

ระบบที่ทำได้สามารถช่วยป้องกันไม่ให้คนที่ไม่มีสิทธิ์ตั้งค่าปรับแต่งสำคัญของ เครื่องบินออกไปได้ง่าย ๆ ผ่าน โปรแกรมควบคุมทั่วไป และยังทำให้ไฟล์ log ที่อยู่ใน SD card อ่านค่ากลับตรง ๆ ไม่ได้ ถ้าไม่มีกุญแจถอดรหัสที่เก็บไว้บนคอมพิวเตอร์ ภายนอก

งานนี้ยังเป็น **PoC สำหรับทดสอบแนวทาง** ไม่ใช่ชุดพร้อมใช้งานจริงในสนามบินหรือ บนเครื่องจริงทันที

ปัญหาที่ต้องการแก้

เดิมที ถ้ามีคนเสียบสาย USB หรือเข้าถึงไฟล์ log ใน SD card ได้ เขาอาจตั้งค่า ปรับแต่งสำคัญของเครื่องออกไปได้ เช่นค่าจูนหรือค่าที่เป็นความลับของบริษัท

PoC นี้แก้ปัญหา 2 จุดหลัก:

1. ป้องกันการตั้งค่ากลับผ่าน MAVLink ถ้ายังไม่ได้ยืนยันตัวตน
2. ป้องกันการเปิดอ่านไฟล์ log ใน SD card แบบตรง ๆ

บริบทของการทดลอง

การทดสอบนี้ใช้ `LOG_BITMASK` เป็นพารามิเตอร์ตัวแทน เพราะเป็นพารามิเตอร์จริงที่มี อยู่ใน SITL และมีค่าให้ตรวจสอบได้ชัดเจน

`LOG_BITMASK` ไม่ใช่ค่าจูนลับจริงของบริษัท แต่เหมาะสำหรับ PoC เพราะใช้พิสูจน์ พฤติกรรมได้ครบ:

- อยู่ในรายการพารามิเตอร์จริงของระบบ
- อ่านค่าได้จริง
- พยายามเปลี่ยนค่าได้จริง
- ถูกเขียนลง log จริง
- ตรวจสอบในไฟล์ `.BIN` ได้ว่ามีหรือไม่มีข้อความ `LOG_BITMASK`

พูดง่าย ๆ คือ ใช้ `LOG_BITMASK` เป็น "ตัวแทนค่าลับ" เพื่อทดสอบระบบป้องกันก่อน เปลี่ยนเป็นรายชื่อพารามิเตอร์ลับจริงในขั้น production

นอกจากนี้ยังมีชื่อ placeholder สำหรับพิสูจน์ระบบแยกชื่อและ prefix:

- `TSV_SECRET_GAIN`
- `TSV_TUNE_`

สองค่านี้เป็นชื่อทดสอบ ไม่ใช่ค่าจูน production

สิ่งที่ทำได้แล้ว

1. ช้อนค่าลับจากคนที่ไม่มีสิทธิ์

เมื่อเปิดโหมดป้องกัน คนที่เชื่อมต่อเข้ามาแบบไม่ผ่านการยืนยันตัวตนจะมองไม่เห็น พารามิเตอร์ที่ถูกกำหนดว่าเป็นค่าลับ

ตัวอย่างค่าที่ใช้ใน PoC:

- LOG_BITMASK
- TSV_SECRET_GAIN
- กลุ่มค่าที่ขึ้นต้นด้วย TSV_TUNE_

2. คนที่มีสิทธิ์ยังดูค่าได้

ถ้าเชื่อมต่อกับ MAVLink2 signing ที่ถูกต้อง ระบบยังอนุญาตให้ดูค่าลับได้ตามปกติ
พุดง่าย ๆ คือ ไม่ได้ลบค่าทิ้ง แค่อันไม่ให้คนที่ไม่มีสิทธิ์เห็น

3. กันการแก้ค่าลับจากคนที่ไม่มีสิทธิ์

ถ้าคนที่ไม่มีสิทธิ์พยายามเปลี่ยนค่าลับ ระบบจะไม่ยอมให้เปลี่ยน ค่าเดิมยังอยู่ เหมือนเดิม

4. เข้ารหัสไฟล์ log ใน SD card

ไฟล์ log ที่เขียนลง SD card ถูกเข้ารหัสไว้ ทำให้เปิดไฟล์ดิบแล้วอ่านค่าลับตรง ๆ ไม่ได้

ต้องใช้เครื่องมือถอดรหัสบนคอมพิวเตอร์ภายนอก พร้อมกุญแจส่วนตัว จึงจะเปิดไฟล์ log กลับมาเป็นไฟล์มาตรฐานของ ArduPilot ได้

5. กุญแจส่วนตัวไม่ได้อยู่ในเครื่องบิน

กุญแจส่วนตัวที่ใช้ถอดรหัส log ไม่ได้ถูกใส่ไว้ใน firmware และไม่ได้อยู่บนเครื่องบิน

เครื่องบินมีแค่กุญแจสาธารณะสำหรับเข้ารหัสเท่านั้น ส่วนกุญแจส่วนตัวต้องเก็บไว้ กับผู้ดูแลบนคอมพิวเตอร์ภายนอก

ผลการทดสอบ

ทดสอบทั้งหมดบน SITL ซึ่งเป็นระบบจำลองของ ArduPilot

ผลรวม:

- Build ผ่าน
- SITL เปิดทำงานได้
- คนที่ไม่มีสิทธิ์มองไม่เห็นค่าลับ
- คนที่มีสิทธิ์มองเห็นค่าลับ
- คนที่ไม่มีสิทธิ์เปลี่ยนค่าลับไม่ได้
- ไฟล์ log ดิบใน SD card ไม่มีข้อความค่าลับให้อ่านตรง ๆ
- ถอดรหัส log ด้วยเครื่องมือ offline แล้วเปิดอ่านด้วยเครื่องมือ ArduPilot ได้
- เมื่อปิดโหมดป้องกัน ระบบกลับไปทำงานเหมือนเดิม

ตารางผล:

หัวข้อ	ผล
Build และเปิด SITL	ผ่าน
ซ่อนค่าลับจากผู้ไม่มีสิทธิ์	ผ่าน
ผู้มีสิทธิ์ยังอ่านค่าลับได้	ผ่าน
กันการแก้ค่าลับจากผู้ไม่มีสิทธิ์	ผ่าน
เข้ารหัส log	ผ่าน
ถอดรหัส log กลับมาอ่านได้	ผ่าน
ปิดโหมดแล้วกลับสู่พฤติกรรมเดิม	ผ่าน

ค่าจริงที่ได้จากการทดสอบ

ส่วนนี้คือค่าที่เห็นจริงจากการรัน PoC ไม่ใช่แค่คำว่า "ผ่าน"

เรื่องที่ทดสอบ	ก่อน / สิ่งที่ยพยายามทำ	หลัง / ผลจริง
Build แบบเปิดระบบป้องกัน	เปิด AP_PARAM_PROTECTION_ENABLED=1 และ AP_LOGGER_ENCRYPTION_ENABLED=1	Build ผ่านสำหรับ SITL
รายการพารามิเตอร์เมื่อปิดระบบป้องกัน	ผู้ใช้ทั่วไปเห็นพารามิเตอร์ LOG_BITMASK ได้	เห็นทั้งหมด 1430 รายการ (received=1430, advertised=1430)
รายการพารามิเตอร์เมื่อเปิดระบบป้องกัน แต่ยังไม่ยืนยันตัวตน	ผู้ใช้ทั่วไปขอดูรายการพารามิเตอร์ทั้งหมด	เห็น 1429 รายการ (received=1429, advertised=1429) และไม่เห็น LOG_BITMASK
ผู้ใช้ที่ยืนยันตัวตนแล้ว	ใช้ MAVLink2 signing แล้วขอดูรายการพารามิเตอร์	เห็นครบ 1430 รายการ (received=1430, advertised=1430) รวมถึง LOG_BITMASK
การอ่านค่าลับแบบไม่มีสิทธิ์	ขออ่าน LOG_BITMASK โดยไม่ signed	ไม่ส่งค่า PARAM_VALUE กลับมา
การพยายามแก้ค่าลับโดยไม่มีสิทธิ์	พยายามเปลี่ยน LOG_BITMASK จาก 65535.0 เป็น 65536.0	ค่าไม่เปลี่ยน ยังเป็น 65535.0
ไฟล์ log ดิบเมื่อเปิดระบบเข้ารหัส	เปิดไฟล์ .BIN ตรงจากดิสก์ ขนาด 1,039,287 bytes	เจอหัวไฟล์ APLOGE1 และค้นหา LOG_BITMASK, TSV_SECRET_GAIN, TSV_TUNE_ ไม่เจอ
การถอดรหัส log	เอาไฟล์เข้ารหัส .BIN ไปถอดด้วยกุญแจ offline	ได้ไฟล์ ArduPilot .BIN ปกติ ขนาด 583,871 bytes จาก 14,227 frames
การอ่าน log หลังถอดรหัส	เปิดไฟล์ที่ถอดแล้วด้วย mavlogdump.py	อ่านได้ปกติ (return code = 0) และพบข้อมูล FMT จำนวน 195 บรรทัด
การยืนยันว่าไม่ได้ลบค่าออกจาก log	ตรวจไฟล์ที่ถอดรหัสแล้ว	พบ LOG_BITMASK หลังถอดรหัส แปลว่าข้อมูลถูกเก็บไว้ แต่ถูกซ่อนไว้ด้วยการเข้ารหัส
log เมื่อปิดระบบป้องกัน	สร้าง log แบบ stock behavior ขนาด 623,239 bytes	เปิดไฟล์ดิบแล้วพบข้อความ LOG_BITMASK ตามปกติ

สรุปแบบง่าย:

- เติมนคนทั่วไปเห็น LOG_BITMASK ได้
- หลังเปิดระบบป้องกัน คนทั่วไปไม่เห็น LOG_BITMASK
- คนที่มีสิทธิ์ยังเห็น LOG_BITMASK
- ถ้าคนไม่มีสิทธิ์พยายามเปลี่ยนค่า ระบบไม่ยอมเปลี่ยน
- log ดิบเปลี่ยนจากไฟล์ที่ค้นหาข้อความลับเจอ เป็นไฟล์เข้ารหัสที่ค้นหาไม่เจอ
- เมื่อนำ log เข้ารหัสไปถอดด้วยกุญแจ offline จะได้ไฟล์ .BIN ปกติกลับมาอ่านได้

สิ่งที่ยังไม่ใช่การใช้งานจริง

PoC นี้ยังไม่ได้ทดสอบกับบอร์ดจริงหรือ Mission Planner/QGC บนสาย USB จริง

ยังไม่ได้ทำเรื่องเหล่านี้:

- Secure boot
- Signed bootloader
- RDP / option bytes
- การล๊อคบอร์ดจริงระดับ production

ส่วนนี้ตั้งใจไว้เป็นขั้นตอนที่ต้องทำโดยคน เพราะมีความเสี่ยงสูง และบางขั้น ย้อนกลับไม่ได้

สิ่งที่ต้องตัดสินใจก่อนใช้จริง

ก่อนนำไปใช้จริง ต้องกำหนดเรื่องต่อไปนี้:

1. รายชื่อพารามิเตอร์จริงที่จะป้องกัน
2. จะใช้กุญแจยืนยันตัวตนแบบเดียวกันทั้ง fleet หรือแยกต่อเครื่อง
3. จะออกกุญแจเข้ารหัส log ต่อ fleet หรือแยกต่อเครื่อง
4. ใครเป็นคนเก็บกุญแจส่วนตัวสำหรับถอดรหัส log
5. ขั้นตอนทดสอบบนบอร์ดจริง
6. ขั้นตอน secure boot และ RDP สำหรับ production

สรุปสำหรับผู้บริหาร

PoC นี้พิสูจน์แล้วว่าแนวทางนี้ทำได้:

- ป้องกันการดึงค่าจูนสำคัญผ่านการเชื่อมต่อที่ไม่มีสิทธิ์
- ป้องกันการอ่านค่าลับจากไฟล์ log ทับใน SD card
- ยังสามารถกู้ log กลับมาอ่านได้ เมื่อผู้ดูแลมีกุญแจถอดรหัส
- ไม่แตะส่วนอันตรายอย่าง bootloader, RDP, option bytes

ข้อสรุปคือ แนวทางนี้พร้อมไปต่อขั้นทดสอบกับ hardware จริง แต่ยังไม่ควรมองว่า เป็น production-ready จนกว่าจะผ่านการทดสอบบนบอร์ดจริงและกำหนดนโยบายกุญแจให้ชัดเจน